A Short History of Soft Fork Activation

Sjors Provoost — sjors@sprovoost.nl

May 26, 2022

This originally appeared as chapter 12 in the book *Bitcoin: A Work In Progress* (btcwip.com).

The Taproot soft fork was activated on November 13, 2021, approximately one year after the finalized code was merged.[1] It happened much more quickly than SegWit did, and with far less drama, but it wasn't an uneventful year. This chapter discusses how soft forks were activated in the past, what options were considered for Taproot, and how Taproot was finally activated.

Ep. 3

We dedicated five episodes to this topic, and the QR codes are placed at various points in this chapter. However, it's far from a one-to-one mapping; they're not even in chronological order.

Soft Forks: A Primer

As Taproot's deployment grew close, the question of how to activate soft forks once again became a topic of debate in the Bitcoin community.

Soft forks, if you recall, are changes to the protocol that are backward compatible. In other words, anyone who has upgraded will reap the benefits of new changes, but those who don't upgrade will still find their software working.

In addition to introducing new features, a soft fork can be used to get rid of bugs and potential vulnerabilities — at least, some of them. The way this is done is by making the rules stricter, but without suddenly freezing anybody's coins.

A simple example of such stricter rules is BIP 66,[2] which mandated that any new signatures had to conform to a strict standard, whereas there was previously some (unintended) flexibility in how to encode signatures.[3]

It may seem paradoxical that strict rules allow for *more* features, but in chapter 3, under the future SegWit versions section, we explained why this works. In this chapter, we're less concerned with how soft forks work, and instead we focus on how they're activated.

There are a few different ways to introduce a soft fork. You can do it randomly by accident, or you can deliberately sneak one into the code. You can also announce a date (known as a flag day) or block height from which the new rules apply. Finally — and this is how things currently work — you can have miners signal and have the soft fork activate once a certain threshold is reached.

But perhaps what matters more than the mechanics of activation is how a decision is reached to deploy the soft fork in the first place. And who decides anyway?

The Earliest Soft Forks

Although the term didn't yet exist in the early days, there were many soft forks, mostly related to closing security holes in the early prototype.[4] In 2013, there was even an accidental soft fork, and in 2015, there was a near-miss accidental soft fork due to OpenSSL changes (see chapter 4).

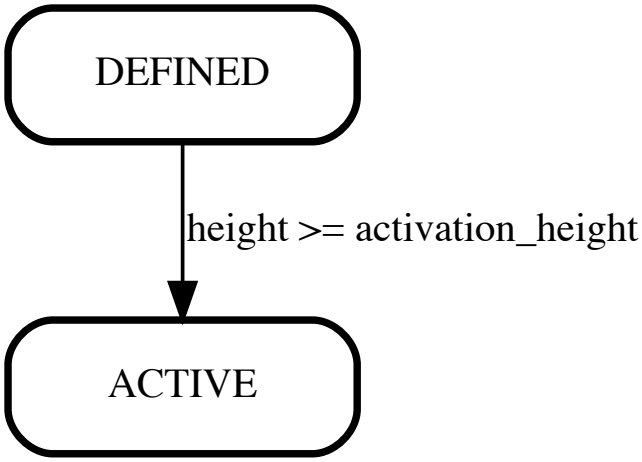
The earliest soft forks mostly used a block height as their method of activation — in other words, "as of this future block, the new rule shall apply." Ideally this is announced well in advance, giving people plenty of time to upgrade. For a "secret" soft fork, developers might simply insist that people upgrade and then explain the reason afterward.

[1] https://github.com/bitcoin/bitcoin/pull/19953

[2] https://en.bitcoin.it/wiki/BIP_0066

[3] https://technicaldifficulties.io/2020/07/22/bip-66-unpacking-der-signatures/

[4] https://blog.bitmex.com/bitcoins-consensus-forks/



Informal diagram of a flag height-activated soft fork

Probably the most infamous soft fork of all time is the one-megabyte block size limit introduced by Satoshi in 2010.[5] The soft fork was released on September 7, 2010, and its `activation_height` was set to 79,400, which occurred just a week later.[6].

Not only was Satoshi's decision to impose this limit a unilateral one, but he did it secretly. He probably found it safer to keep this change under wraps because he didn't want to alert potential attackers to a gaping security hole, wherein massive blocks could have ground the network to a halt.[7]

It's not that nobody looked at the source code changes, because in the forum where Satoshi announced this new release, there was discussion of *another* soft fork at that same height. In any case, nobody paid much attention to it until many years later, when this reduced limit became a practical issue in the form of increasing fees for scarce block space.

But barring some existential emergency, there's a general consensus that this isn't an acceptable way of introducing a soft fork now. If there had been a debate on the block size limit back then, perhaps the drama that came later could've been prevented.

How Is a Soft Fork Enforced?

Releasing a new software version that activates a soft fork at a given height is one thing. But unless the right people run it and do so in time, it won't actually take effect. If a soft fork is released in a forest…

What Satoshi did was announce the new version on a forum and presume the community was small enough that everyone would update well before the activation height, even if that was only a week away. This was generally not put to the test, because many of those initial soft forks were made in such a way, either by design or accident, that only a malicious actor would produce blocks that violate the new rule, and there weren't many of them around.

Despite that, even in 2010, there was a growing understanding that the safest way to enforce a soft fork is to have a supermajority of miners run the latest version. This is because nodes will follow the longest chain that they consider valid. So even if a user hasn't upgraded their node, as long as the majority of miners are enforcing the rules, they'll produce the longest chain, and the user's node will follow along. This way, even if a miner maliciously or accidentally produces an invalid block, the majority of miners won't build on

[5] Since the very first release of version 0.1.0 on January 9, 2009, there has been a 32MB limit (`MAX_SIZE`) that applies to various things. This includes the block size, which was checked in `CheckBlock()`. See https://satoshi.nakamotoinstitute.org/code/. Then, on July 15, 2010, Satoshi introduced `MAX_BLOCK_SIZE=1000000` and changed the miner software to not produce blocks larger than that in https://github.com/bitcoin/bitcoin/commit/a30b56ebe76ffff9f9cc8a6667186179413c6349. So far, no soft fork. It was just a change to the software used by miners, which they could've reverted without producing invalid blocks. Months later, on September 7, he modified a related function, `AcceptBlock()`, to enforce `MAX_BLOCK_SIZE` https://github.com/bitcoin/bitcoin/commit/f1e1fb4bdef878c8fc1564fa418d44e7541a7e83. This was the actual soft fork, and it was released the same day in v0.3.12. Both commits pretended to do completely unrelated things. Nowadays, code reviewers frown upon commits that touch anything outside the area they claim to change — even if just removing a blank line.

[6] https://bitcointalk.org/index.php?topic=999.msg12181

[7] Back in 2017, I ran an experiment where I took older versions of Bitcoin Core and measured how long it took them to catch up to the present-day blockchain. Modern nodes did this many times faster, thanks to various improvements over the years (see e.g. chapter 5). But more importantly, Bitcoin Core v0.5, released in 2012, was completely unable to keep up with the chain. It would just crash or grind to a halt for weeks. Without the block size limit introduced by Satoshi, anyone back in 2012 could've produced huge blocks that then would've overwhelmed the available node software (see appendix A).

top of it, and the invalid block goes stale.[8]

We don't know how many individual miners there were in 2010, but it's conceivable they were very much on top of these updates.[9] And there wasn't any money to be made from mining: The first pizza sale was only in May of that year.[10] Even in 2013, when an accidental soft fork happened, enough miners deployed the fix in just a few hours.[11] But emergencies aren't the same as regular soft forks.

Who Decides?

Initially, Satoshi would unilaterally decide what to change, though he ultimately couldn't force anyone to run the new versions he released, so he couldn't make completely arbitrary controversial changes.[12] Nowadays, without going too deep in the weeds, Bitcoin development follows a process of "rough consensus," as described in RFC 7282.[13]

Anyone can propose a change to the rules of Bitcoin. But they not only need to convince others of the usefulness; they also need to address all technical objections that are raised to it. For example, if someone complains that a proposal would break their (wallet) software, the proposal author can't simply say "tough luck." Instead, they have to address the issue. Maybe they can propose a simple fix for the wallet in question, or they can modify their own proposal so it doesn't break stuff.

This — along with a few other requirements — usually involves a lot of back and forth on technical mailing lists, as well as many iterations of improving the proposal. Many proposals don't survive this process at all, because it turns out they cause too many problems.

Hard fork proposals often get rejected based on just the fact that they both break existing software and require every participant to upgrade at the same time. A soft fork variant of the same proposal would address both concerns, so it's generally preferred instead. This is why Bitcoin Core developer Luke Dashjr's offhand suggestion that SegWit could be implemented as a soft fork was such a game changer.[14]

In addition to not having any unaddressed objections, there's also the need to get enough experienced developers to review a proposal. Lack of enthusiasm among a very small group of such experienced developers can cause a perfectly fine soft fork to never see the light of day. Or, more often, a lack of reviewer enthusiasm combined with difficult-to-address technical problems will keep the proposal in limbo.

But if all goes well and the code ends up merged into the Bitcoin Core software, there's still the matter of what activation procedure to apply. This is subject to the same kind of rough consensus discussion as a proposal; people may love a proposed soft fork but object to a flag day activation for all the reasons explained above. To avoid scenarios where proposals get stuck in a discussion about their activation, ideally the community agrees on a single activation mechanism that's applied for every soft fork. But, well, that turns out to be a challenge.

Signaling (BIP 9)

So if a flag day or block height isn't the best way to activate soft forks, how can we do better? One idea was to have miners signal readiness in the blocks they create. Initially, this was done by increasing the block version number.[15] Signaling works as a coordination mechanism for the network to figure out that enough miners have upgraded.

[8] See chapter 7 for an explanation of stale blocks.

[9] Some speculate that a single miner nicknamed Patoshi, not necessarily Satoshi, controlled more than 50 percent of the hash rate until February 2010: https://whale-alert.medium.com/the-satoshi-fortune-e49cf73f9a9b

[10] https://bitcoinmagazine.com/culture/the-man-behind-bitcoin-pizza-day-is-more-than-a-meme-hes-a-mining-pioneer

[11] https://bitcoinmagazine.com/technical/bitcoin-network-shaken-by-blockchain-fork-1363144448

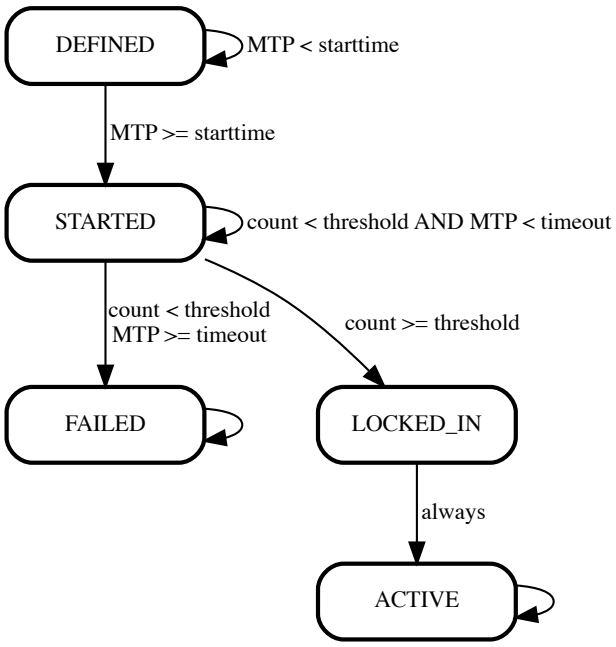
[12] It's best to leave the party while you're still having fun, and perhaps he did just that: "But as frustrations with his authority and availability built, it became all too common for users to decry Satoshi the admin, Satoshi the bottleneck, Satoshi the dictator." https://bitcoinmagazine.com/technical/what-happened-when-bitcoin-creator-satoshi-nakamoto-disappeared

[13] https://datatracker.ietf.org/doc/html/rfc7282

[14] https://news.ycombinator.com/item?id=11230394

[15] BIP 34 https://en.bitcoin.it/wiki/BIP_0034 in 2012 used version 2, and BIP 66 in 2015 used version 3. Once 95 percent of recent blocks contained this new version, the new rules would apply.

The mechanism was improved and further formalized in BIP 9.[16] As part of the signaling mechanism embedded in the code, there's a starting date (`starttime`) when miners begin signaling, and a deadline (`timeout`) at which point they give up if the `threshold` wasn't reached. Tallying happens in rounds of 2,016 blocks, or about two weeks. If the threshold is reached in any round that ends before `timeout`, the new rules are active. This is easy for every node in the network to verify.



BIP 9 flow

The significance of 2,016 is that it's the number of blocks in a single difficulty adjustment period, or retarget period.[17] In the diagram above, each arrow represents one signaling period. The looping arrows indicate when the state stays the same — for example, when a soft fork is `DEFINED` (meaning the node knows about it, but there's no signaling yet), it'll stay that way if the MTP[18] is still below `starttime`. When it's at or after `starttime`, the state jumps to `STARTED`. It stays there pending signaling. For each period — say every two weeks — we check if enough blocks are signaling. If so, we move to the next phase, which is `LOCKED_IN`. If not, and if `timeout` is reached, we move to `FAILED`. `LOCKED_IN` is a grace period where the new rules don't yet apply, but after two weeks, the soft fork is `ACTIVE` and the rules do apply.

Signaling hands control of upgrade activation to miners for a predefined period. It requires a signaling readiness of 95 percent for a soft fork activation to succeed.

This mechanism has the additional benefit of allowing the deployment of multiple soft forks in parallel. Each gets assigned a specific signaling bit.

BIP 9 was used successfully to deploy the CSV and SegWit soft forks, and it could've been used for Taproot as well. The first deployment went smoothly, but as we'll soon see, the second one involved much drama and took far longer than many people considered necessary. This led to worries that perhaps BIP 9 isn't a future-proof deployment mechanism.[19] As a result, several other mechanisms were proposed, along with variations on those mechanisms.

Always Verify Blocks!

Unfortunately, no amount of signaling is useful when miners don't actually enforce the new rules. Remember that we need the majority of miners, i.e. the longest chain, to enforce the new rules to protect non-upgraded nodes (which in turn allows these upgrades to remain non-mandatory).

During the BIP 66 soft fork in 2015, it turned out that a large portion of miners, despite signaling readiness, weren't verifying the new rules.[20] As soon a transaction that violated the new rules appeared in a block, those miners failed to reject it and instead kept building on top of it.[21] At the same time, the miners that upgraded and checked the new rules did reject the block. They produced an alternative, valid block at the same height and kept building on top of that new branch. Eventually, their new branch overtook the invalid branch, causing the non-verifying miners to switch over to

[16] https://en.bitcoin.it/wiki/BIP_0009

[17] https://en.bitcoin.it/wiki/Difficulty

[18] MTP stands for Median Time Past, and it refers to the middle block of the last 11 blocks. This is a mechanism used to discourage miners from gaming the timestamp in each block.

[19] Its use of timestamps rather than block heights also seemed to needlessly complicate things.

[20] https://www.reddit.com/r/Buttcoin/comments/6dvkr6/short_writeup_of_the_bip66_disaster_is_this/

[21] https://bitcoin.org/en/alert/2015-07-04-spv-mining#cause

the valid branch. The first time, the invalid chain branched off for six blocks. The next day, it happened again, but only for three blocks, perhaps because more miners upgraded their software as they became aware of the problem.

Skipping block verification is risky for miners even without a soft fork,[22] but under normal and benevolent circumstances, they may get away with it. However, in a soft fork where a majority of miners enforces the new rules, but a minority doesn't, when one of these minority miners accidentally mines an invalid block, other such non-verifying miners will build on top of it. The upgraded majority will ignore all these blocks, and once their chain is longer, all these minority miners find their blocks stale. That's what happened with BIP 66.

The First Drama, and BIP 148/91

When it was time to deploy SegWit, after the code was ready, BIP 9 was used again because it worked fine before. But as months went by, in none of the biweekly retargeting periods was the required 95 percent block signaling threshold reached. Although one can't tell by looking at the blockchain, it appeared that several large miners were using the BIP 9 readiness signal, or lack thereof, as a vote, in turn blocking the upgrade.

They weren't raising any technical objections, so the previously established RFC 7282-style rough consensus was still intact. Was it just inertia and apathy? Was there a secret technical objection they had to the proposal?[23] Or were miners frustrated with and lacking confidence in the developers, perhaps in part due to language and cultural barriers?[24]

In any case, BIP 9 wasn't supposed to be a referendum, and given how quickly miners were able to upgrade with previous soft forks, this stalemate made little sense and frustrated those who wanted to take advantage of the SegWit features we discussed in chapter 3. One particularly frustrating aspect of this situation is that the lack of signaling was holding back the much demanded block size increase that SegWit offered.

At this point, most of the Bitcoin Core developers were perhaps just as frustrated, but they preferred to stay on the conservative side and simply wait for the `timeout` and some earlier success. When there's no consensus on a new course of action, the default is to just do nothing. But that doesn't have to stop anyone else.

Around early March of 2017, there was a group of people that said, "Hey, you know what? It's time for a flag day." They picked August 1, 2017, as a date and said that on that day, their nodes would enforce the new SegWit rules.

To be more precise, on that date, they would require all blocks to signal for SegWit. That signaling would in turn cause SegWit to activate. BIP 148 was the mechanism, and it was commonly referred to as a User-Activated Soft Fork (UASF).[25] The similarity of this acronym to a certain branch of the US military was helpful in its marketing.

The question is: Did it work? One can't tell by just looking at the blockchain. When you look at historical blocks, you just see 95 percent signaled, and the soft fork was activated. Now, it's of course very remarkable that this activation occurred right before August 1 and not some random other date that it could have happened. But there are no stale branches of non-signaling blocks that we could point to as evidence for a fight between miners and UASF nodes.

As we'll explain in the `LOT=true` discussion below, it's probably a good thing that no confrontation in the form of competing blockchain branches happened.

A few days after BIP 148 was published, an alternative to it was posted on the developer mailing list. It proposed activating SegWit, along with an additional doubling of the block size using a hard fork. We already pointed out above that, thus far, hard fork proposals haven't survived the technical objections raised against them, e.g. the need for all node users and wallet software to upgrade at the same time, and the mandatory nature of such

[22] It's even more reckless for miners to not even verify the *transactions* they selected to put in their own blocks. But back in 2015, there was at least one small miner who did this: https://www.reddit.com/r/Bitcoin/comments/3c305f/comment/csrt3dg/

[23] It was speculated that a technique called covert AsicBoost was giving certain miners an advantage over their competitors that they preferred not to disclose: https://blog.bitmex.com/the-blocksize-war-chapter-14-asicboost/

[24] Many miners were Chinese, and many developers were from Western countries.

[25] https://en.bitcoin.it/wiki/BIP_0148

an upgrade.

But despite not being well received in a technical forum, a group of major companies in the space met in a New York hotel and announced the so-called New York Agreement.[26] They also added a lower activation signal threshold.

The Blocksize War (the book mentioned at the end of chapter 3) goes into more detail about where all that led (spoiler: They called off the hard fork at the last minute). For the purpose of this chapter, it's interesting to briefly explain the idea of lowering the activation threshold, which enjoyed a more positive reception.

The next day, BIP 91 was proposed on the mailing list.[27]. It worked as follows: If 80 percent of miners signaled in favor, then (two weeks later) all miners would need to signal. And once all miners signaled, the 95 percent threshold would be reached. This would cause SegWit to activate from the perspective of everyone involved.

Users that run the more conservative Bitcoin Core node would see the 95 percent signal and consider SegWit active. Users that run the more aggressive UASF node would see the same, provided it happened before August 1.

Similar to the situation with UASF, we can't really tell what happened by just looking at blocks.[28]

So in the end, all we can say is it didn't go wrong. Nobody called each other's bluff. But what did come out of it is that everyone realized it was important to rethink how to actually activate soft forks. Having a situation where miners block activation without (publicly sharing) a good technical reason isn't ideal. A bunch of contentious chain splits as different camps battle it out isn't great either — it defeats the purpose of carefully engineering soft forks that maintain a well-functioning blockchain.

Rethinking Activation, and the Introduction of BIP 8

Inspired by UASF, as well seeing the need to clean up BIP 9, BIP 8[29] was proposed.[30] It uses a signaling mechanism similar to BIP 9, but based on block height rather than timestamps. This makes it easier to reason about reorganizations, e.g. when the last block of a signaling period arrives just *before* the `timeout`, a soft fork would activate, but then if an alternate branch of blocks appears and takes over, and this alternate branch arrives (has timestamps of) directly *after* `timeout`, then — on that branch, that rewriting of history — it wouldn't activate.

The downside of using a block height is that you can't predict on which date the `timeout` is going to happen, because it depends on how fast blocks are produced.[31]

So far, BIP 8 would just be a nice drop-in replacement for BIP 9. But where it really differentiates itself is in the UASF-style flag date for forced signaling. This is indicated by the `MUST_SIGNAL` phase in the diagram, which otherwise doesn't differ much from what you saw above with BIP 9.

What this means is that the flag date doesn't activate the soft fork itself. Rather, if, closer to the date, there's a block that isn't signaling support for the soft fork, that block will be rejected:

[26] https://dcgco.medium.com/bitcoin-scaling-agreement-at-consensus-2017-133521fe9a77

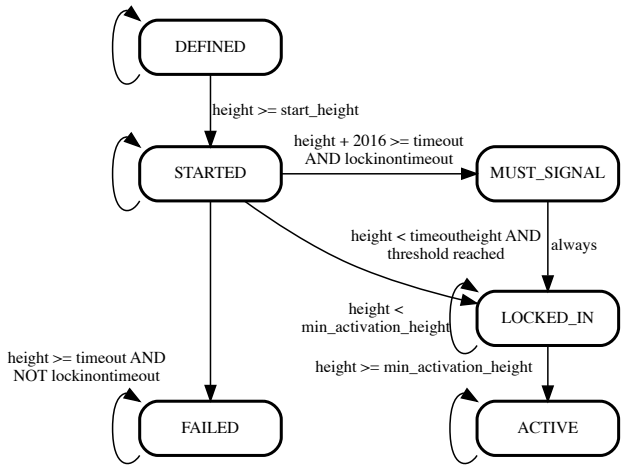
[27] https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2017-May/014380.html

[28] SegWit signaled on bit 1, and the New York Agreement folks signaled on bit 4 (as did BIP 91). The bit 4 signal did cross the required threshold: https://bitcoinmagazine.com/technical/bip-91-has-activated-heres-what-means-and-what-it-does-not. However, that doesn't prove that BIP 91 *caused* SegWit to activate. That's because things happened too quickly. During the retarget period, the BIP 91 status was `LOCKED_IN`, meaning it wasn't yet enforcing its 100 percent signaling requirement. But right in that period, SegWit reached its 95 percent threshold on bit 1. So in the next period, BIP 91 was `ACTIVE` and SegWit was `LOCKED_IN`. Both required 100 percent signaling at that point. In reality, it's likely that miners were simply setting bit 4, but not actually running any of the software that used it.

[29] https://en.bitcoin.it/wiki/BIP_0008

[30] In case you're wondering why a new proposal gets a lower BIP number, my guess is as follows: BIP numbers tend to be thematically grouped in ranges of 10, e.g. BIP 340–343 all relate to Taproot. BIP 1 and 2 are meta issues, and they explain, among other things, how Bitcoin ought to be upgraded. So it makes sense to include soft fork activation logic in the single-digit range, but fill it out in descending order.

[31] This is mainly a problem for developers who use the testnet to test e.g. wallet software ahead of the real activation. On the testnet, blocks don't arrive in semi-regular 10-minute intervals, but can instead arrive in huge numbers. This makes it impossible to pick reasonable activation heights. For a more thorough explanation and solution, see appendix A for the episode about Signet.



BIP 8 flow

That's how it forces signaling toward the end.

Like with the UASF, this forced signaling only makes sense if there are *other* nodes out there that don't have such a flag date.[32] This is why the flag day setting itself is optional. What exactly does optional mean here? It could mean that there are two different downloads: one with the flag day enabled and one without, presumably released by two separate teams with different priorities. It could also mean that there's a single download that allows the user to choose.

This allows for an escalation ladder: Perhaps most of the community starts out not using a flag date, and then as time goes by, more do.

In this sense, it formalizes the process that happened informally in 2017, basically saying: If you want to do something like a UASF, please do it in this specific way.

Just like its predecessor, UASF, this proposal isn't without its problems. The following paragraphs outline how forced signaling is supposed to work.

If you run a node with this feature enabled, then when, after the flag day, a block that doesn't include a signal is produced, your node will consider this block invalid, no matter how many other blocks are built on top of it. If some miners produce an alternative branch of blocks, even if it's shorter, that do include the signal, your node will follow that alternative branch. Eventually, if and only if enough blocks are produced on this alternative branch, the soft fork is guaranteed to activate.

On the other hand, *if you run a node without this feature*, or for that matter, if you run older node software that doesn't know about the soft fork, then you'll continue to follow the longest chain, regardless of what its block signal is. If the longest chain happens to comply with mandatory signaling, your node will follow it. If it doesn't comply, your node will also follow it. The scenario to worry about here is when, initially, the longest chain doesn't signal, but after some time it gets overtaken by a chain that does. Since your node doesn't care if blocks signal or not, it'll happily switch over to this new branch.

In any scenario where two alternative chains exist, it's unsafe for users whose node follows one branch to transact with users whose node follows the other branch. In fact, it's unsafe for *anyone* to use the blockchain at that point. On the other hand, as long as the only chain in existence complies with mandatory signaling, there's nothing to worry about. This might remind some readers of the game theory around mutual assured destruction (MAD).

To Argue a `LOT`

Ep. 29

This setting to require mandatory signaling became known as `LOT`. We dedicated several episodes to the debate around it, not all of which made into this book. The transcript for the accompanying episode can be found here.[33]

When it came to the Taproot activation process, there was a debate surrounding this `LOT` parameter, which stands for Lock-in On Timeout. If you refer to the BIP 8 diagram above, `lockinontimeout` appears in the horizontal arrow. When set to `false` we transition to `FAILED` after the soft fork times out. But when it's set to `true`, then in the very last 2,106-block retargeting

[32] For nodes without that setting, the horizontal line from `STARTED` to `MUST_SIGNAL` is never used, and instead the vertical line from `STARTED` to `FAILED` would occur, and it's essentially BIP 9.

[33] This transcript was written by Michael Folkson. The site contains many other transcripts from technical Bitcoin podcasts, conference talks, and even group conversations. Many of them are written by Bryan Bishop aka Kanzure, who is quite possibly one of the fastest typists on Earth. https://diyhpl.us/wiki/transcripts/bitcoin-magazine/2021-02-26-taproot-activation-lockinontimeout/

period, we go to `MUST_SIGNAL`, and that's always followed by `LOCKED_IN`. In other words, we lock in right before ("on") the timeout, hence the name.

In other words, in the case of both `LOT=false` and `LOT=true`, miners can signal for an upgrade for one year. Then, if the specified threshold percentage — in the case of Taproot, 90 percent — is met, it'll activate. However, if it isn't met, two things could happen. With `LOT=false`, the Taproot upgrade will expire, but a new activation period could be implemented if the community decides to try again with a new software version. With `LOT=true`, nodes will begin only accepting blocks that signal for the upgrade, which forces the activation.

Initially, in early 2021, there wasn't yet a Bitcoin Core release that would activate Taproot. Such a release was still pending community debate on what the appropriate activation mechanism should be. So this is a different context than during the UASF debate in 2017, where there *was* an existing Bitcoin Core BIP 9 deployment in the `STARTED` stage. Meanwhile, in early 2021, the discussion revolved around whether Bitcoin Core should switch from its usual BIP 9 deployment system to BIP 8, and if so, if `LOT` should be set to `true` or `false`.

The switch from BIP 9 to BIP 8 wasn't very controversial, as long as it stuck to the more conservative `LOT=false` incarnation. But it's not a no-brainer, because there's always a risk when making *any* change, especially to something as critical as the code that decides which rules apply to blocks. So it still raised the question: Is a switch to BIP 8 with `LOT=false` worth it?

It might be worth it to remain compatible with software from an independent group that insists on `LOT=true` (compatibility shouldn't be construed as endorsement). But this debate was never settled.[34]

The real controversy revolved around `LOT=true`.

A lot of people supported `LOT=true` because it made it so miners couldn't have a veto. The counterargument to that is that miners don't have a veto anyway; they can merely delay an orderly activation. This is because node owners can always switch to a new version of the node software that has a flag day, bypassing signaling altogether. But they'll have to wait.[35] If activation is delayed by a lack of signaling with `LOT=false`, the upgrade will expire after a year. After that, we could deploy any new upgrade mechanism, and potentially one without any signaling.

Another option could even be to start with `LOT=false`, wait half a year, and then say, "This is taking too long. Let's take a little more risk and set `LOT=true`."

The debate itself arguably ended up slowing down Taproot activation: There was no reason to believe miners would object to Taproot the way some did to SegWit, and there was no political drama around Taproot itself. It was very conceivable that a BIP 9 or BIP 8 with `LOT=false` activation would've gone smoothly, as it happened in the pre-SegWit era. But the debate created a stalemate, because Bitcoin Core generally doesn't ship functionality that's deemed controversial, and at that point, all activation options were controversial to at least some people.

It wasn't a stalemate because some people *preferred* `LOT=true`. Remember the RFC 7282 rough consensus process. As long as they didn't *object* to `LOT=false` (or BIP 9), then their preference for `LOT=true` could be dismissed. Because in that case, what would have remained were objections to `LOT=true` and no objections to `LOT=false`, and so you'd have moved forward with the thing nobody objected to.

But some people went beyond a mere preference for `LOT=true`. They claimed `LOT=false` was unsafe,[36] i.e. they objected to it. So we ended up with two proposals that both had objections. That these objections were advocated for by a very small number of developers was immaterial. They needed to be addressed, e.g. shown to be incorrect

[34] There was pull request that implemented a transition from BIP 9 to BIP 8 in Bitcoin Core. This is a generic transition and not Taproot specific. However, it contained `LOT=true` code, which added complexity and triggered objections. A pure `LOT=false` version might have made it through review. https://github.com/bitcoin/bitcoin/pull/19573

[35] If the community doesn't wait until the timeout and activates a soft fork with a flag day, then anyone who doesn't upgrade to the new flag day software will think the soft fork failed to activate. Mandatory signaling avoids this need to wait, but at a cost.

[36] https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2021-February/018498.html

or solved somehow. And that process could and did drag on for a while.

The Chain Split Scenario

The main objection to `LOT=true` was the same as the one raised against UASF: It could cause a chain split. Remember the reference above to MAD. An often-heard argument *for* `LOT=true` is that a chain split is so terrible — especially for miners who wouldn't be able to sell their new coins, it won't happen. They believe this is sufficient deterrence for miners to simply signal. Such game theory is beyond the scope of this book, but we can clarify what such a chain split would be like and why it's bad.

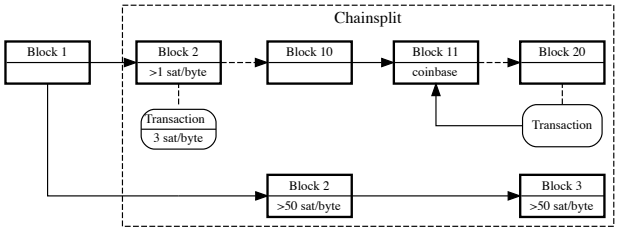
The following is one such example of how, if one part of the network runs `LOT=true` and another part runs `LOT=false` or an older version of the software, bad things could happen in the form of a chain split.

Let's say you're running the `LOT=true` version of Bitcoin Core and you want Taproot to activate. But the scenario here is that the rest of the world — including most miners — isn't doing this. The day arrives and you see a block that isn't signaling correctly but you want it to signal correctly, so you say "This block is now invalid, so I'm not going to accept this block. Instead, I'm just going to wait until another miner comes with a block that does meet my criteria." Maybe that happens once in every 10 blocks, so you're seeing new blocks, but they're coming in very, very slowly.

So far, you might think this is merely annoying. But now let's put some money on the line. What if you're trying to receive a payment?

Let's say somebody who runs a node with `LOT=false` sends you 1 BTC. So in this scenario, they're on a branch that's growing 10 times faster than the branch your node is seeing. Let's also say their blocks aren't completely full, so the sender uses a low fee rate. The transaction confirms quickly in their branch. But you're on this shorter, slower-moving branch, and all those transactions have to be squeezed into fewer blocks. Those blocks are completely full. In your branch, the transaction doesn't confirm. It's just sitting there in the mempool.

[37] https://bitcoin.stackexchange.com/a/89418/4948



In this example, blocks on the fast moving side of the chain split require a fee of at least 1 satoshi per vbyte.[37] Due to congestion on the slower moving side, its minimum fee rate is 50 sat/vbyte. A transaction paying 3 sat/vbyte only confirms on one side of the split.

That's actually a relatively benevolent scenario. You've learned that you shouldn't accept unconfirmed transactions. You'll have a disagreement with your counterparty, you'll say, "It hasn't confirmed," and they'll say, "It has confirmed." Assuming you're aware of this chain split, you might realize what's going on.

If you had anticipated this situation, you would've asked your counterparty to pay a much higher fee than their node suggests (and maybe deduct it from the amount). Otherwise, you could've used something called Child Pays For Parent (CPFP) by taking their unconfirmed low fee transaction and spending it back to yourself with a very high fee transaction. The combined fee is then enough for miners to include both.

A much worse scenario is when your counterparty is trying to send you coins that don't exist on your branch. How can that be? One possibility is that they're spending a coin that descends from a coin that was created by a miner on their branch. Every block contains a coinbase transaction, which sends the block subsidy (6.25 BTC at the time of writing), plus any transaction fees collected, to the miner. Each branch will have a different sequence of miners producing its blocks. This means each branch has unique coinbase transactions that don't exist on the other branch. That, in turn, means any transaction spending from such a coinbase transaction can't exist in the other branch.[38]

The general phenomenon described here is called transaction replay. In the case of a hard

[38] This can be used to generate something so called UTXO Fairy Dust. By deliberately spending a piece of that dust in a transaction, you can guarantee it won't replay on the other branch.

fork, it's often desirable to *prevent* transactions on one side of the fork from appearing (being replayed) on the other side. If this sounds fascinating, then you may like the author's presentation on the topic.[39] Otherwise, just understand that whether you want to prevent replay or guarantee it, it's a pain.

It's possible that the coins on both branches will have a different market price. In that case, the above examples become even more complicated, because you and your counterparty can't agree on what 1 BTC is worth.

In any case, translated to the RFC 7282 rough consensus process: If you propose something that creates the possibility of transaction replay, you should address how to deal with it.

But it gets worse.

Continuing with the above scenario of a short branch with `LOT=true` and a long branch with `LOT=false`, perhaps over time, the market price for coins on `LOT=true` increases. This price increase attracts miners, and this increase in mining activity increases the pace of block creation on this branch. In turn, it slows it down on the `LOT=false` branch.

This can lead to a tipping point if `LOT=true` overtakes the `LOT=false` branch. Your node chugs along just fine. It was following the `LOT=true` branch already when it was shorter, and it continues to do so now that it's longer.

But your friend in the other branch is about to have a very traumatic experience. Their node detects the longer branch. From its point of view, that branch is perfectly valid; the mandatory signaling on it doesn't violate any rules. So it's going to switch over!

This is very bad. Any transactions your friend had sent or received on their branch, if they don't also appear on your branch, will disappear. More accurately: Those transactions will either return to mempool, from which they could later end up getting confirmed again, or they could disappear entirely if they descend from a coinbase transaction.

Anything like a big reorganization[40] will cause mayhem, even without any malicious actors. Let's say a miner deposited coins on an exchange and your friend withdrew some coins from that same exchange. Exchanges generally don't allocate specific coins to specific users, so there's a chance the exchange used some of the miner coins to pay your friend. That means your friend never received that money and has to complain to the exchange. But if this happens on a large enough scale, the exchange is probably going to be insolvent.

Again translated to the RFC 7282 rough consensus process: Is it really enough to simply claim this scenario won't happen because of incentives to prevent it? Is it so unlikely that not even a contingency plan is needed to handle it? Some cities have nuclear shelters despite the MAD game theory, though others have indeed repurposed them as shopping malls. It seems like more of a political debate than a technical discussion.

Finally, it's worth pointing out that all the problems that `LOT=false` users are subjected to in world with `LOT=true` clients are also encountered by users who don't upgrade at all. Avoiding mandatory upgrades is also something to consider.

`LOT=true` Client, Rogue?

Ep. 36

This time around, the first software download release with the Taproot activation code wasn't Bitcoin Core. Instead, two developers decided to independently release a modified version of Bitcoin Core, which included BIP 8 and the `LOT=true` behavior.[41]

With open source software, anyone is free to release any variation of the software they want. Similarly, everyone is free to download whichever variation they want. However, in addition to the general objections to `LOT=true` above, there are other practical matters to think about when downloading such an alternative implementation. We cover these in the episode above. In particular, it's important to make sure you're not accidentally downloading malware (see chapter 9).

The Speedy Trial Proposal

[39] https://sprovoost.nl/2017/11/10/a-short-history-of-replay-protection-2bd8b288cf94/

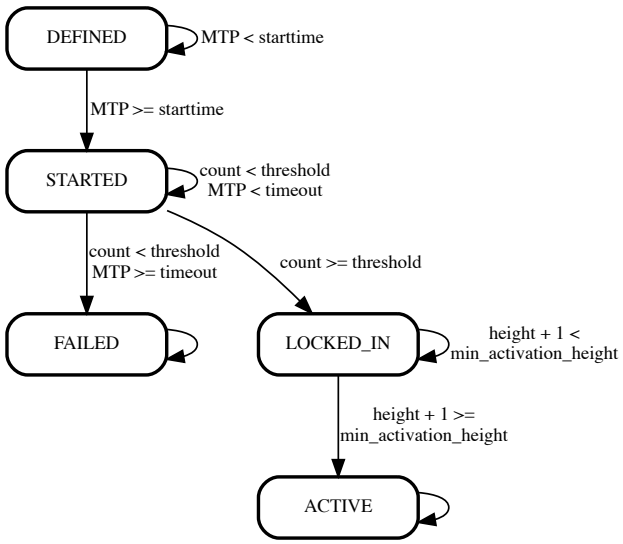
[40] https://en.bitcoin.it/wiki/Chain_Reorganization

[41] https://www.reddit.com/r/Bitcoin/comments/mruopv/bitcoincorebased_bip8_lottrue_taproot_activation/

Ep. 31

To get out of this stalemate, Speedy Trial came to the rescue. It proposed[42] the following: "Rather than discussing whether or not there's going to be signaling and having lots of arguments about it, let's just try it quickly." The proposed timeline[43] suggested the signaling would start in early May, last three months (until August), and then be activated three months later, in November.

Spoiler: As we mentioned at the beginning of the chapter, Taproot indeed activated on November 13, 2021.



Speedy trial flow.

The diagram above is quite simple to understand. Compared to BIP 9 above, it only introduces one new concept: a minimum activation height. This adds a delay in the transition from `LOCKED_IN` to `ACTIVE`.

Although it's conceptually almost the same as BIP 9, the dates picked for Taproot are quite different than what would've been picked before. The waiting period before signaling (`DEFINED`), as well as the signaling period (`STARTED`), are much shorter than usual (months instead of a full year). This way, we could know the result faster.

Knowing the result quickly is great, but the rush runs the risk of miners using fake signals rather than actually upgrading. So to add a margin of safety, the transition from `LOCKED_IN` to `ACTIVE` was increased from the usual single period (two weeks) to a fixed block height, which was expected to be reached in November 2021. That was the only code change required (a much smaller change than BIP 8).

So the "speedy" part refers to figuring out miner readiness, or at least to figuring out if there was any previously unknown miner objection, or just apathy. The rest of the process was slower, and it behaved a bit more like a flag day. Once the signal threshold was reached, the soft fork was set in stone, meaning it would happen, at least if people ran the full nodes.

This process made it so Taproot would activate six months after the initial release of the software, assuming 90 percent of miners were signaling. If that threshold wasn't met, the proposal would've expired, and activation options would've been discussed more, albeit with more data to back up decision making.

Speedy Trial seemed to sufficiently address the objections to BIP 9. From the objectors' point of view, because it was so fast, their own plans for BIP 8 wouldn't be delayed.

With the controversy (temporarily) out of the way, more developers came out of the woodwork and started writing code that could actually get Speedy Trial done.[44] In turn, because there were more developers from different angles cooperating on it and getting things done a little bit more quickly, it demonstrated that Speedy Trial was a good idea. When you have some disagreement, then people start procrastinating, not reviewing things, or not writing things. But if people begin working on something quickly and it's making progress, that's a vague indicator that it was a good choice.

We Have Taproot `LOCKED_IN`!

Ep. 40

Bitcoin Core v0.21.1 with the Speedy Trial code was released on May 1, 2021.[45]

[42] https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2021-March/018583.html

[43] https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2021-March/018594.html

[44] Mainly https://github.com/bitcoin/bitcoin/pull/21377, https://github.com/bitcoin/bitcoin/pull/21686, and a BIP 8-based alternative that was briefly considered: https://github.com/bitcoin/bitcoin/pull/21392

[45] https://bitcoincore.org/en/2021/05/01/release-0.21.1/

The first retargeting period started a week before that release on April 24, 2021, and the threshold wasn't reached. The second retargeting period also didn't reach the threshold, but the third time was a charm. The 90 percent signaling threshold was reached on June 12, 2021, with `LOCKED_IN` happening a few days later.[46] It lasted until the November activation.

Remember that the signal for a soft fork (BIP 9, BIP 8, or Speedy Trial) is just a bit flag in the block header. Miners can and do use custom software to set this bit. At the same time, miners run full nodes that actually enforce the consensus rules. But if they don't upgrade their own nodes, then their outdated nodes will simply ignore the flag, and their nodes won't enforce the new rules. For that to happen, they need to actually upgrade their node software.

In general, it's preferred if miners actually upgrade their nodes and don't fake signal. That's one reason why the timeout in BIP 9 was so long. But because Speedy Trial happened on such short notice, some may have considered it too risky to upgrade their software. Others ran into practical issues performing the upgrade. Mining pool operator Alejandro De La Torre described some of the practical issues he encountered in the field on a podcast episode.[47]

The accompanying episode goes into further detail about what, once Taproot activation became inevitable, needed to happen before it could ultimately be used on the Bitcoin network safely. We also explain how upcoming Bitcoin Core releases will handle the Taproot upgrade, especially with respect to its wallet software. At the time of writing, there's some basic Taproot wallet support, but it's still a work in progress.

Moving Forward

Because Speedy Trial was successful, it's possible we can use it as a template for soft fork activation moving forward. Or, we could interpret the lack of drama as an argument to just stick with BIP 9 or a `LOT=false` version of BIP 8. Perhaps some aspects of `LOT=true` deployment can be made safer.

Even if it's inherently unsafe, it could make sense to continue developing it further, having the code already in place in case it's ever needed. Perhaps the Bitcoin Core software could have generic support for it, even if the project itself recommends against using it. The best time to think about such matters is when they're not yet needed.

Burying Soft Forks

Ep. 54

After all is said and done and a soft fork has activated, what do you do with the activation code? Is it merely a scaffold that can be removed once the new rules are active? Or is the activation mechanism itself a permanent part of the rules?

As was done with previous soft forks, it looks like a future Bitcoin Core release will "bury" the Taproot activation. This means the node will treat the Taproot rules as if they've been active since Bitcoin's very beginning. This is possible because, when applying these rules retroactively, only one historical block does not conform to them. This block can be grandfathered in.[48]

In the episode we explain what the benefits are of burying a soft fork, in particular pointing out how it helps developers when they review the Bitcoin Core codebase or when they perform tests on it.

After that, we outline a potential edge case scenario where burying soft forks could, in a worst-case scenario, split the Bitcoin blockchain between upgraded and non-upgraded nodes. Bitcoin Core developers generally don't consider this edge case — a very long block re-org — to be a realistic problem and/or believe that this would be such a big problem that a buried soft fork would be a minor concern comparatively. However, as we explain, not everyone agrees with this assessment entirely.

[46] https://sports.yahoo.com/locked-bitcoin-taproot-upgrade-gets-120837972.html

[47] https://stephanlivera.com/episode/277/

[48] https://github.com/bitcoin/bitcoin/pull/23536